

Nesneye Yönelik Programlama

Hafta 6

Örnek 1

Fan Sınıfı: Bir fanı temsil etmek için **Fan** adlı bir sınıf tasarlayınız. Sınıf aşağıdaki özellikleri içermelidir:

- Fan hızını belirtmek için **SLOW, MEDIUM ve FAST** adlı üç sabit tanımlayınız. Bu sabitlerin değerleri sırasıyla **1, 2 ve 3** olmalıdır.
- Fanın hızını belirten **speed** adında private bir int veri alanı tanımlayınız (Varsayılan değeri: **SLOW**).
- Fanın açık olup olmadığını belirten **on** adında private bir boolean veri alanı tanımlayınız (Varsayılan değeri: **false**).
- Fanın yarıçapını belirten **radius** adında private bir double veri alanı tanımlayınız (Varsayılan değeri: **5**).
- Fanın rengini belirten **color** adında bir String veri alanı tanımlayınız (Varsayılan değeri: **blue**).
- Bu dört veri alanının her biri için **getter (erişimci)** ve **setter (değiştirici)** metotlarını yazınız.
- Parametre almayan (**no-arg**) bir constructor tanımlayınız. Bu constructor varsayılan bir fan nesnesi oluşturmalıdır.
- **toString()** adında bir metot yazınız.
 - Eğer fan **açık**sa, metot fanın hızını, rengini ve yarıçapını tek bir metin olarak döndürmelidir.
 - Eğer fan **kapalı**ysa, metot fanın rengini ve yarıçapını, ayrıca **“fan kapalı”** ifadesiyle birlikte tek bir metin olarak döndürmelidir.

Örnek 1

İstenenler:

1. Sınıfı Java dilinde **gerçekleyiniz (implement ediniz)**.
2. Bir test programı yazınız:
 - İki adet Fan nesnesi oluşturunuz.
 - **1. nesne için:**
 - hız: maksimum (FAST)
 - yarıçap: 10
 - renk: sarı
 - durum: açık
 - **2. nesne için:**
 - hız: orta (MEDIUM)
 - yarıçap: 5
 - renk: mavi
 - durum: kapalı
3. Nesneleri **toString() metodu** ile ekrana yazdırınız.

```

public class Fan {

    // Sabitler
    public static final int SLOW = 1;
    public static final int MEDIUM = 2;
    public static final int FAST = 3;

    // Veri alanlari (fields)
    private int speed;
    private boolean on;
    private double radius;
    private String color;

    // Default constructor
    public Fan() {
        speed = SLOW;
        on = false;
        radius = 5;
        color = "blue";
    }

    // Getter - Setter metodlari
    public int getSpeed() {
        return speed;
    }

    public void setSpeed(int speed) {
        this.speed = speed;
    }

    public boolean isOn() {
        return on;
    }

    public void setOn(boolean on) {
        this.on = on;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

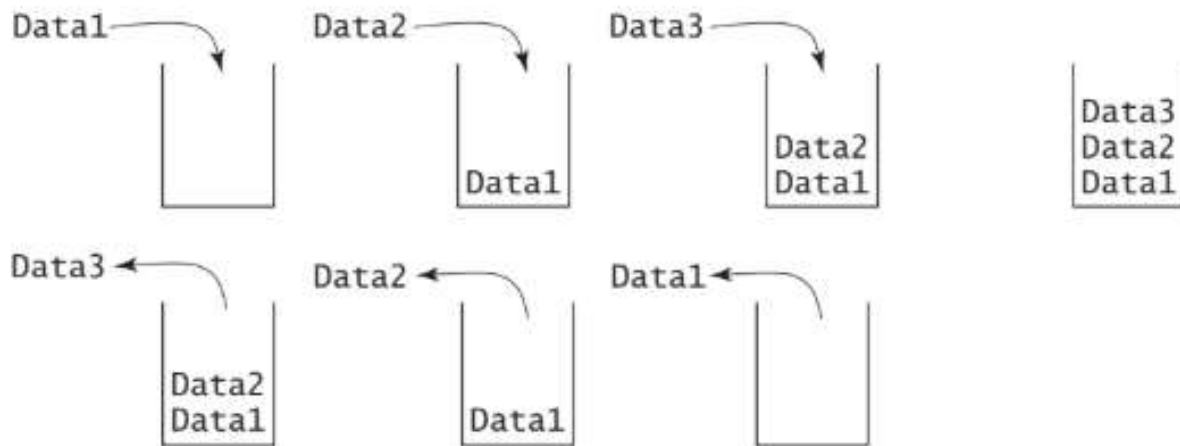
    // toString metodu
    public String toString() {
        if (on) {
            return "Speed: " + speed +
                ", Color: " + color +
                ", Radius: " + radius;
        } else {
            return "Fan is off, Color: " + color +
                ", Radius: " + radius;
        }
    }
}

```

```
public class Main {  
  
    public static void main(String[] args) {  
        // 1. Fan  
        Fan fan1 = new Fan();  
        fan1.setSpeed(Fan.FAST);  
        fan1.setRadius(10);  
        fan1.setColor("yellow");  
        fan1.setOn(true);  
  
        // 2. Fan  
        Fan fan2 = new Fan();  
        fan2.setSpeed(Fan.MEDIUM);  
        fan2.setRadius(5);  
        fan2.setColor("blue");  
        fan2.setOn(false);  
  
        // Yazdirma  
        System.out.println(fan1.toString());  
        System.out.println(fan2.toString());  
  
    }  
}
```

Örnek 2

Stack (Yığın) Sınıfı: Stack (yığın), verileri **son giren ilk çıkar (LIFO – Last In First Out)** mantığıyla saklayan bir veri yapısıdır.



Örnek 2

- Java dilinde, sadece int türünde değerleri tutan **StackOfIntegers** adlı bir sınıf tasarlayınız. Sınıf aşağıdaki özellikleri içermelidir:
- Stack içerisindeki elemanları tutmak için elements adında int[] türünde bir dizi tanımlayınız. Stack'teki mevcut eleman sayısını tutmak için 'size' adında int türünde bir veri alanı tanımlayınız.
- Parametresiz (no-arg) bir constructor yazınız. Bu constructor, kapasitesi **16** olan boş bir stack oluşturmalıdır.
- Parametre alan (capacity: int) bir constructor yazınız. Bu constructor, verilen kapasiteye sahip boş bir stack oluşturmalıdır.
- empty() adında bir metot yazınız. Bu metot, stack boş ise true, değilse false döndürmelidir.
- peek() adında bir metot yazınız. Bu metot, stack'in en üstündeki elemanı **silmeden** döndürmelidir.
- push(int value) adında bir metot yazınız. Bu metot, verilen değeri stack'in üstüne eklemelidir.
- pop() adında bir metot yazınız. Bu metot, stack'in en üstündeki elemanı kaldırmalı ve geri döndürmelidir.
- getSize() adında bir metot yazınız. Bu metot, stack'teki eleman sayısını döndürmelidir.

Örnek 2

İstenenler:

- Sınıfı Java dilinde gerçekleyiniz (implement ediniz).
- Bir test programı yazınız:
- Bir adet StackOfIntegers nesnesi oluşturunuz.
- Stack içerisine sırasıyla **0, 1, 2, ..., 9** sayılarını ekleyiniz.
- Stack boşalana kadar elemanları çıkararak ekrana yazdırınız.
- Program çalıştırıldığında sayılar **ters sırada** ekrana yazdırılmalıdır.
- Örnek çıktı: 9 8 7 6 5 4 3 2 1 0

StackOfIntegers
<code>-elements: int[]</code> <code>-size: int</code>
<code>+StackOfIntegers()</code> <code>+StackOfIntegers(capacity: int)</code> <code>+empty(): boolean</code> <code>+peek(): int</code> <code>+push(value: int): void</code> <code>+pop(): int</code> <code>+getSize(): int</code>

An array to store integers in the stack.
The number of integers in the stack.

Constructs an empty stack with a default capacity of 16.
Constructs an empty stack with a specified capacity.
Returns true if the stack is empty.
Returns the integer at the top of the stack without removing it from the stack.
Stores an integer into the top of the stack.
Removes the integer at the top of the stack and returns it.
Returns the number of elements in the stack.


```

public class StackOfIntegers {

    private int[] elements;
    private int size;

    // Default constructor (kapasite 16)
    public StackOfIntegers() {
        this(16); // Aynı sınıftaki başka bir constructor çağırılır
    }

    // Parametrelili constructor
    public StackOfIntegers(int capacity) {
        elements = new int[capacity];
        size = 0;
    }

    // Stack boş mu?
    public boolean empty() {
        return size == 0;
    }

    // Üstteki elemanı silmeden getir
    public int peek() {
        if (empty()) {
            throw new RuntimeException("Stack boş!");
        }
        return elements[size - 1];
    }

    // Eleman ekle
    public void push(int value) {
        // Eğer doluysa kapasiteyi artır
        if (size >= elements.length) {
            int[] newArray = new int[elements.length * 2];
            System.arraycopy(elements, 0, newArray, 0, elements.length);
            elements = newArray;
        }

        elements[size++] = value;
    }
}

```

```

// Eleman çıkar
public int pop() {
    if (empty()) {
        throw new RuntimeException("Stack boş!");
    }
    return elements[--size];
}

// Eleman sayısı
public int getSize() {
    return size;
}
}

```

```
package btu;

public class Test {

    public static void main(String[] args) {
        StackOfIntegers stack = new StackOfIntegers();

        // 0'dan 9'a kadar ekle
        for (int i = 0; i < 10; i++) {
            stack.push(i);
        }

        // Stack boşalana kadar çıkar ve yazdır
        while (!stack.empty()) {
            System.out.print(stack.pop() + " ");
        }

    }

}
```